eedi-mining-misconceptions-in-mathematics

1位 詳細Solution

Rank	1
Team	MTH 101
Author	Raja Biswas
Topic ID	551688
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551688
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

この興味深いコンペを開催してくださったKaggleとEEDIに感謝します。挑戦しがいのあるコンペで、新しい手法を学び、応用する機会がたくさんありました。いつものように、革新的なideaと活発な議論を提供してくれるKaggle communityにも感謝しています。私の詳細なsolutionを共有できることを嬉しく思います。皆さんの役に立てば幸いです。

1 Task

診断用の数学MCQが、その正解と1つの不正解とともに与えられ、2.5k+のmisconceptionのpoolから上位25個のmisconception（不正解との関連度の高い順にソート）を推薦するのがタスクでした。 標準的なretrieve-rerankの枠組みが自然に当てはまりましたが、いくつかの課題がありました。

- 現在のLLMは、(1) `Novice Learner`、すなわち特定のmisconceptionから生じる不正解を生成すること、あるいは (2) `Expert Tutor`、すなわち与えられた不正解を説明するmisconceptionを特定すること、のどちらを模倣するのも苦手な場合が多いです。LLMは数学の問題を解くのは得意ですが、counterfactual reasoningはそれほど得意ではありません。
- misconceptionのpoolには、密接に関連した概念上の誤りと計算上の誤りが混在しており、微妙な違いを正確に見分ける高い識別能力が求められます。この点はコンペのoverviewでも強調されていました。

> 事前学習済み言語モデルを使った初期の試みは、おそらく問題に含まれる数学的内容の複雑さのために、うまくいきませんでした。

- このタスク設定では、既知のmisconceptionでうまく機能するだけでなく、新しいmisconceptionにも汎化することがmodelに求められます。

2 Meta Thoughts

コンペ開始時点で、良いsolutionには何が重要になりうるかについて、次の仮説を持っていました。

- 合成データはout-of-distributionへの汎化に役立つはず
- 数学の領域は客観的に検証・curationできるので、高品質な合成データの生成は実現可能なはず
- 大規模LLMはreasoningが格段に優れているので、良いteacher（複数でも）からのdistillationは非常に強力なはず
- 最上位のLLM（例: Claude 3.5 Sonnet）からChain of Thoughts (CoT)をdistillすれば、難しい例に対処するのに役立つはず
- quantizationの手法とcalibration datasetが重要になりうる。top 1 accuracyを維持するために、quantizeしたmodelにrecovery adapterを追加する必要があるかもしれない。
- LoRAよりfull trainingの方が良いかもしれない

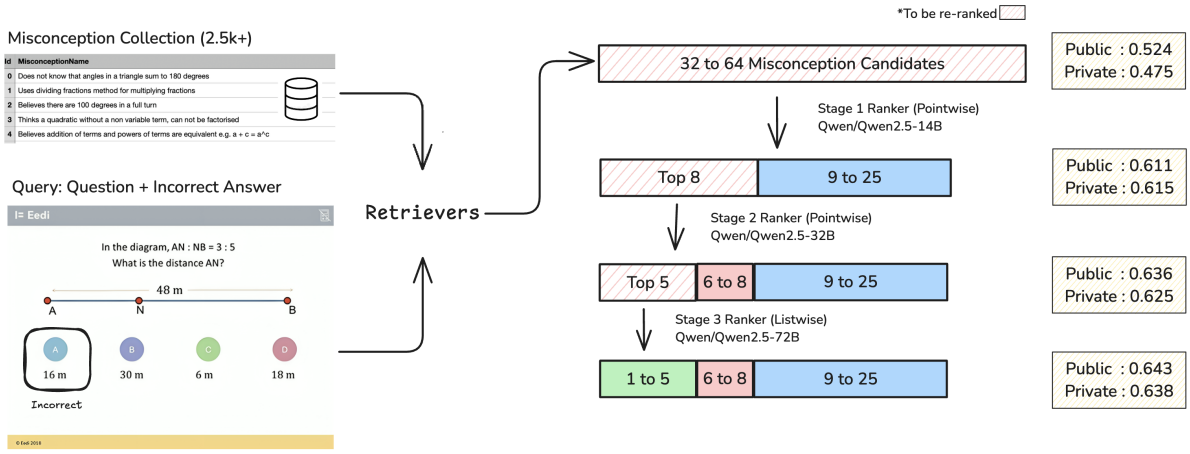
これらの仮説がコンペを通して実験を設計する指針となり、最初の試みがうまくいかなくても1つのideaを粘り強く追い続ける助けになりました。

3 Pipeline

私は、次の4ステップの流れからなるretrieve-and-rerankシステムを実装しました。

- **Retrieval**: MCQと不正解の各組み合わせについて、上位のmisconception候補を特定します。具体的には、retrieveした上位32候補を保持し、さらにdynamic thresholdに基づいて最大32個の候補を追加しました。dynamic thresholdは `top candidate similarity score - constant` として計算します。similarity scoreがdynamic thresholdより高い候補を残します。
- **14B Ranker**: fine-tuneした `Qwen/Qwen2.5-14B` rankerで、retrieveした全候補を処理し、上位8個に絞り込みます。
- **32B Ranker**: fine-tuneした `Qwen/Qwen2.5-32B` rankerで、上位8候補を処理し、上位5個に絞り込みます。
- **72B Ranker**: fine-tuneした `Qwen/Qwen2.5-72B` rankerで、上位5候補の最終的な順位を決めます。

pipelineの模式図と、各stageのleaderboard scoreを以下に示します。



最終的な25個のmisconception予測は、次のように構成しました。

- 1〜5番目の候補は72B rankerから
- 6〜8番目の候補は32B rankerから
- 9〜25番目の候補は14B rankerから

4 Data

retrieverとrankerの両modelの学習データには、コンペのデータ（1.8k例）と合成データ（10k例）を混ぜて使いました。合成データは、素の性能と、未知のmisconceptionに対する汎化能力の両方を改善するうえで決定的な役割を果たしました。

4.1 Synthetic Data

理想的には、合成MCQは次の性質を備えているべきです。

- 正解として指定された答えが、本当に正解である（高いaccuracy）。

- 不正解が、対象のmisconceptionから直接生じている（高い診断力）。
- misconception pool（2.5k+）にあるすべてのmisconceptionの中で、指定されたmisconceptionが、与えられた不正解を最も正確に説明する（高い分解能）。
- 生成されたMCQが、幅広いsubjectとconstructをカバーしている（高い多様性）。

最初は、期待するformatと対象のmisconceptionを指定するだけの単純なpromptで診断用MCQを生成しました。しかし、その結果できたdatasetは、程度の差こそあれ、ほぼすべての性質に反していました。これは予想どおりでした。なぜなら、

- 単純なpromptには1つのmisconceptionに関する情報しか含まれていませんでした。そのため、データを生成するLLMは、misconception pool内の密接に関連するmisconceptionをまったく把握していませんでした。
- few-shot exampleがないため、期待される品質、難易度、語調、言葉遣いの基準がありませんでした。
- 先進的なLLMでさえ、misconceptionやcounterfactual reasoningを効果的に扱うのは苦手です。
- curationを一切行っていませんでした。

これらの問題に対処するためにいくつかの戦略を試し、`grouped synthetic data generation`というアプローチとLLMによるfilteringを組み合わせることで、高品質な合成datasetを作ることができました。具体的には、まずvalidation dataに対するretriever/rankerの予測におけるmisconceptionの共起統計を利用して、密接に関連するmisconceptionのclusterを作りました。clusteringの過程はこのnotebookで示しています。misconception clusterの例を以下に示します。

```
- Thinks x^2 - a^2x is a valid form of difference of two squares
- When factorising a quadratic without a non variable term, tries to double bracket factorise
- Incorrectly factorises a quadratic
- Believes the constant in an expanded quadratic comes from adding the two numbers in the brackets
- Believes the coefficent of x in an expanded quadratic comes from multiplying the two numbers in the
brackets
- Does not recognise difference of two squares
- Does not realise a quadratic must be in the form ax^2+bx+c=0 to be factorised
- Does not know how to identify common factors from algebraic terms
- Believes the difference of 2 squares method of factorising also works for the addition of 2 squares
- When factorising, believes they can choose different factors for each term
- Believes they can factorise a difference of two squares by placing the constant in both brackets
without square rooting
```

次に、clusterに含まれるmisconceptionについての参照例を4〜5個含めた改良版promptを使い、`Claude 3.5 Sonnet`で各clusterにつき5個の新しいMCQを生成しました。prompt engineeringには、Anthropic cookbookのmetapromptツールを使いました。データ生成に使った最終的なpromptは次のとおりです。

```
You will be generating Multiple Choice Questions (MCQs) that diagnose specific mathematical
misconceptions. Here are the misconceptions you should focus on:

<misconceptions>
{cluster_misconceptions}
</misconceptions>

Here are reference MCQs that demonstrate how to effectively diagnose these misconceptions:

<reference_mcqs>
{reference_mcqs}
</reference_mcqs>

Your task is to generate {num_mcqs} new MCQs that diagnose misconceptions not already covered by the
reference MCQs.
```

```
First, analyze the reference MCQs carefully:
1. For each reference MCQ, identify in your <analysis> tags:
   - Which misconception it targets
   - How the incorrect answers map to specific misconceptions
   - What makes the question effective at diagnosing the misconception
2. Note the style, difficulty level, and precision of language used

Then, in your <planning> tags:
- List which misconceptions still need coverage
- For each needed misconception, brainstorm mathematical contexts where it commonly appears
- Design questions where the misconception leads naturally to specific wrong answers
- Take notes on how you can craft new MCQs that adheres to the reference MCQs' style, difficulty
level, and precision of language

Finally, generate new MCQs following these important guidelines:
- Make sure each incorrect answer maps clearly to exactly one misconception
- Use precise mathematical language matching the style of reference MCQs
- Make questions challenging enough that students must demonstrate real understanding
- Ensure wrong answers are plausible and stem from genuine misconceptions, not careless errors
- Use the exact wording of misconceptions as given in the misconceptions list
- Pay careful attention to subtle differences between the misconceptions and observe which one is the
most appropriate for a given incorrect answer
- Keep the construct name and subject name as short as possible hiding the details of the
misconception
- Questions should be of higher difficulty level than reference MCQs
```

4.1.1 Curation

低品質な合成MCQを除外するために、LLM-as-a-judgeを使いました。`GPT-4o`に、次のようにして合成MCQの品質を評価させました。

```
You will analyze how well an incorrect answer reflects a suspected misconception in a mathematics
problem. Your goal is to determine whether there is a clear, logical connection between the
misconception and the wrong answer.

Here is the problem with both correct and incorrect answers. The suspected misconception is also
provided:
<problem>
{PROBLEM_DATA}
</problem>

First, analyze the problem in your scratchpad:
<scratchpad>
1. Solve the problem independently to verify the correct answer
2. Examine how someone holding the suspected misconception would approach the problem
3. Trace the logical path from misconception to incorrect answer
4. Identify any gaps or inconsistencies in this connection
</scratchpad>

Then provide your evaluation using this format:
<evaluation>
1. Brief explanation of how the misconception could lead to the wrong answer
2. Score from 0-10 based on these criteria:
   - 10: Perfect alignment - wrong answer is direct result of misconception
   - 8-9: Strong alignment - clear logical path from misconception to answer
   - 5-7: Moderate alignment - connection exists but has some gaps
   - 1-4: Weak alignment - connection is unclear or requires assumptions
   - 0: No alignment - misconception does not explain wrong answer
</evaluation>
```

```
Important guidelines:
- Focus solely on the logical connection between misconception and wrong answer
- Do not speculate about other possible misconceptions
- Be specific about how the misconception leads to the error
- Flag and deduct scores if any assumptions are required to connect misconception to answer
- Consider whether a student with this misconception would consistently arrive at this wrong answer
```

judgeに`GPT-4o`を選んだのはvibe testに基づいてです。scoring schemeに対してより敏感で、かつClaude 3.5 Sonnetと同程度に一貫していると感じました。

4.1.2 Incorporating Additional Misconceptions

合成データを作る過程で、LLMはhostが提供したmisconception poolに含まれないmisconceptionを多数（4k+）生み出しました。retrieverとrankerのmodelの汎化能力を高めるため、これらの外部misconceptionを残すことにしました。

外部misconceptionを既存のものと単純にmergeすると、衝突やノイズを持ち込んでいたでしょう。多くは既存のmisconceptionの言い換えにすぎなかったからです。これを慎重に扱うため、次の手順を採用しました。

- 文字列の正規化（小文字化、末尾の句読点の除去など）によって既存のmisconceptionとの照合を試みる
- embedding model（`thenlper/gte-base`）のsimilarity scoreを使い、非常に高いthreshold（0.995）で既存のmisconceptionとの照合を試みる
- 既存のmisconceptionのいずれかとかなり高いsimilarity（0.95〜0.995）を持つ新しいmisconceptionを除去する
- 残った外部misconceptionはそのまま残す

4.1.3 Dataset

最終的なdatasetはここにアップロードしています: Eedi - Mining Misconceptions in Mathematics。内容は次のとおりです。

- コンペのMCQ 1.8k + 生成したMCQ 10.6k
- 4791個のmisconception

形式は元のコンペdatasetと同じです。

参考: answer.aiによる合成データ生成の記事 How To ~~Train~~ Synthesize Your ~~Dragon~~ Data をぜひ読むことをおすすめします。

4.2 Chain of Thought (CoT) Generation

先進的なLLMのreasoning能力をsolution pipelineにdistillするため、まず`Claude 3.5 Sonnet`が生成した思考過程を含むdatasetを作りました。Claudeには、(i) 問題文、(ii) 正解、(iii) 不正解、(iv) ground truthのmisconception、(v) 密接に関連するいくつかのmisconception、を与えました。これらの情報を使い、生徒が不正解を選ぶに至ったであろう思考過程を生成するようClaudeにpromptを与えました。promptを以下に示します。

```
You will analyze a student's incorrect answer to identify the specific reasoning flaw that led to
their error.
Your goal is to explain precisely how their misconception caused them to arrive at the wrong answer.

Here is the problem information:
<problem_data>
```

```
# Question: Simplify the expression: \[x \cdot y \cdot x\]
# Correct Answer: \(x^2y\)
# Incorrect Answer: \(x^2\)
# Primary Misconception: Ignores variables without explicit coefficients when multiplying
</problem_data>

Here are related misconceptions that are similar but do not explain this specific error as precisely:
<related_misconceptions>
- Thinks only like terms can be multiplied
- Fails to combine all instances of the same variable
- Incorrectly identifies an incomplete variable factor
- Does not understand how to multiply algebraic terms
</related_misconceptions>

First, examine all components of the problem carefully:
1. The problem statement and question asked
2. The correct answer and solution method
3. The student's incorrect answer
4. The primary misconception given
5. The related misconceptions that should be distinguished from the primary one

Then, reconstruct the student's likely thought process:
- Identify the exact point where their reasoning diverged from the correct solution path
- Note which specific mathematical operations or concepts they misapplied
- Connect their error directly to the stated primary misconception
- Verify that this explanation better fits the error than the related misconceptions

Write your analysis in <evaluation> tags, following this structure:
- Show the correct calculation first
- Show the incorrect calculations that demonstrate the error
- Explain the specific flaw in the student's reasoning
- Demonstrate how the misconception led to this particular error
- Distinguish from the related misconceptions
- Keep your explanation to 5-6 clear, non-repetitive sentences
- Focus solely on the reasoning that produced this specific error

Guidelines for writing your explanation:
- Do not restate the problem or name the misconception
- Be precise about the mathematical concepts involved
- Show exactly how the misconception led to the error
- Distinguish from related misconceptions
- Avoid repetition
- Stay focused on this specific error
```

このdatasetを使って、Qwen 2.5 seriesの3つのmodel（`Qwen/Qwen2.5-Math-7B`、`Qwen/Qwen2.5-14B`、`Qwen/Qwen2.5-32B`）をreasonerとしてfine-tuneしました。注目すべき点として、fine-tuningの入力からは正解のmisconceptionと関連misconceptionを除外しました。inference時、これらのmodelが見るのは (i) 問題文、(ii) 正解、(iii) 不正解、です。そのうえで、生徒が不正解を選んだ背後にあるであろう思考過程を生成します。inference時のCoT生成には`vllm`を使い、設定は`temperature=0.7, top_p=0.8, repetition_penalty=1.0, max_tokens=256`としました。

CoT datasetはここにアップロードしています: Eedi - CoT Dataset。

5 Train-Validation Split

コンペのdatasetを5 foldに分割しました。最初は`QuestionId`フィールドに基づいて分割しましたが、これは過度に楽観的な推定となり、validation scoreとleaderboard scoreの間に大きなgapが生じました。次に`SubjectId`でGroupKFoldを試しましたが、今度は過度に悲観的な推定になりました。最終的に`ConstructId`での

GroupKFoldに落ち着きました。これによりValidation/LBのgapが小さくなり、public LB scoreが~0.62の範囲まで良い相関が得られました。

開発中は、常にfold 1-4でmodelを学習し、fold 0でvalidationしました。合成データはfold 99として扱い、training setに含めました。最終modelは、開発段階と同じhyperparameterを使い、dataset全体で学習（full fit）しました。

6 Retrievers

retrieverのfine-tuneには標準的なMultipleNegativesRankingLossを使い、学習scriptの開発ではFlagEmbeddingのcodebaseを参考にしました。この段階では`map@25`と`recall@32`の両方のmetricを追跡し、`recall@32`の最適化を優先しました。pipeline全体の性能にとってはrecallの方が重要だったからです。

いくつかのretrieverをfine-tuneし、scoreは次のとおりでした。

		Validation		Leaderboard	
Model	**Backbone**	**MAP@25**	**Recall@32**	**LB (Public)**	**LB (Private)**
1	intfloat/e5-mistral-7b-instruct	0.476	0.921	0.466	0.417
2	BAAI/bge-en-icl	0.493	0.909	0.463	0.419
3	Qwen/Qwen2.5-14B	0.486	**0.937**	0.449	0.451
4	Qwen/Qwen2.5-14B (re-ranker distillation)	**0.528**	0.926	**0.493**	**0.456**

training batchにhard negativeを組み込むことと、re-rankerのscoreをdistillすることは、`map@25`の性能を一貫して改善しましたが、`recall@32`には良い影響を与えませんでした。最終submissionの選択では、`map@25`が最も良いencoder（Model 4）ではなく、recallの高い`Qwen/Qwen2.5-14B` encoder（Model 3）を選びました。同じ理由で、BGE encoder（Model 2）も除外しました。ちなみに、encoderのみを使った私の最良submission（Model 1 + Model 2 + Model 4）は、public LBで0.524、private LBで0.475でした。

retrieverはLoRAでfine-tuneし、設定は`r=64, alpha=128, learning_rate_lora_a=1e-5, learning_rate_lora_b=5e-5, LoRA on all linear layers, batch_size=128, and epochs=12`としました。recall性能を改善したいくつかの重要な要素は次のとおりです。

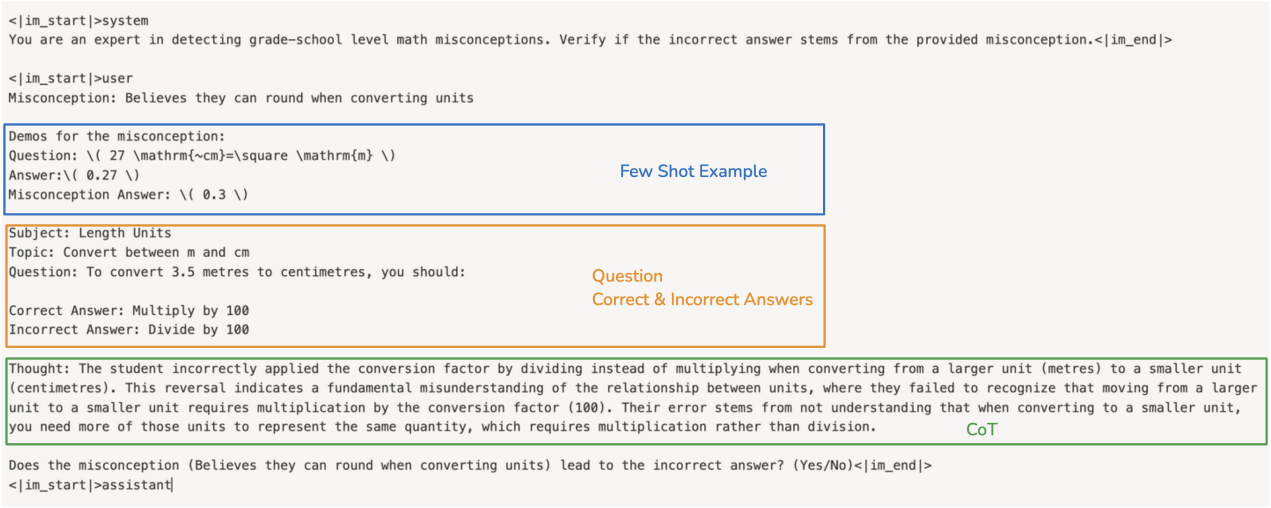
- temperatureを、LLMベースのencoderで一般的に使われる0.02ではなく、0.01に設定したこと
- 各training batchにmisconceptionごとのdemonstrationが1つだけ現れるようにしたこと。複数のdemonstrationがあると、in-batch negativeを通じてlabelノイズが入り込むためです
- `Qwen/Qwen2.5-14B` encoderを、利用可能なすべての合成データ（4.1.1節のcuration前のもの）でpretrainingしたこと

うまくいかなかったこと:

- 反復的なhard negative mining
- cross-device negativeによるbatch sizeの拡大
- 独自のbatching戦略。例えば、同じSubjectIdの (query, misconception) 正例ペアを同じbatchに入れるなど
- nvidia/NV-Embed-v2で使われている戦略と同様に、LLM retrieverをbi-directional encoderへ変換しようとした私の試み

6 Re-rankers

Qwen 14B rankerが、retrieveしたすべてのmisconception（32〜64候補）を処理して上位8候補を特定するという、重い仕事の大半を担いました。このmodelはpointwiseのアプローチで学習しており、context windowの中で一度に1つのmisconceptionだけを見ます。modelの入力は次のように構成しました。



logitは、次のように'Yes'と'No'のtoken scoreの差を取って計算しました。

```
outputs = self.model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=True)
logits_yes = outputs.logits[:, -1, self.yes_loc]  # Yes token logit at the last position [bs]
logits_no = outputs.logits[:, -1, self.no_loc]  # No token logit at the last position [bs]
logits = logits_yes - logits_no  # [bs]

logits = logits.reshape(-1, self.group_size)
labels = labels.to(logits.device).reshape(-1)
ce_loss = self.loss_fn(logits, labels)
```

各batchには、与えられたQuestion-Incorrect Answerの組み合わせについて、1個の正例misconceptionと`batch_size-1`個の負例misconceptionを含めました。labelは正例misconceptionのindexとし、学習にはcross entropy lossを使いました。modelはLoRAで学習し、設定は`r=64, alpha=128, LoRA on all linear layers`としました。fine-tuning中、LLM（`Qwen/Qwen2.5-14B`）のlanguage modelling head（`lm_head`）はfreezeしました。

6.1 Ablations

rerankerの性能とその汎化能力を高めるうえで、次の4つの戦略が特に重要であることがわかりました。

6.1.1 Few Shot Examples

LLMのin-context learning能力を保持して活用するため、model入力のcontextに参照用のmisconception例を任意で数個（0〜2個）含めました（上の図に示したとおりです）。すべての学習例がこれらのfew-shot demonstrationを含んでいたわけではありません。狙いは、参照例があるときはそれを使い、ないときは自身の内部reasoningに頼るようmodelを促すことでした。inference時には、training set全体から1つまたは2つの例をdemonstrationとして使いました。このアプローチは14B rankerの性能に特に効果的で、private LB scoreを0.495から0.531へ押し上げました **(+0.036)**。この時点では、学習にはコンペのデータのみを使っていました。

6.1.2 Distillation / Pseudo Labelling

次に、合成例（curation後）をpseudo labelingを通じて学習pipelineに組み込みました。まず、コンペのdatasetのみを使って2つのpointwise 72B model（`Qwen/Qwen2.5-Math-72B`と`Qwen/Qwen2.5-72B`）をfine-tuneしました。次に、これらのmodelを使って合成例のpseudo labelを生成しました。最後に、コンペのデータ + pseudo labelを付けたデータを使って、次のiterationの14B rerankerをfine-tuneしました。この戦略により、14B rerankerのprivate LB scoreは0.531から0.575へ向上しました **(+0.044)**。

6.1.3 Negative Ratio

次に、正例あたりの負例の数を増やす実験をしました。学習中にrankerへ大量の負例を見せることが性能向上に役立ちました。各正例について、負例の数を以前の16から24に増やしました。このとき、合成例の数も2倍にscaleしました。これら2つの変更により、14B rerankerのprivate LB scoreは0.575から0.596へ向上しました **(+0.021)**。

6.1.4 Chain of Thought (CoT)

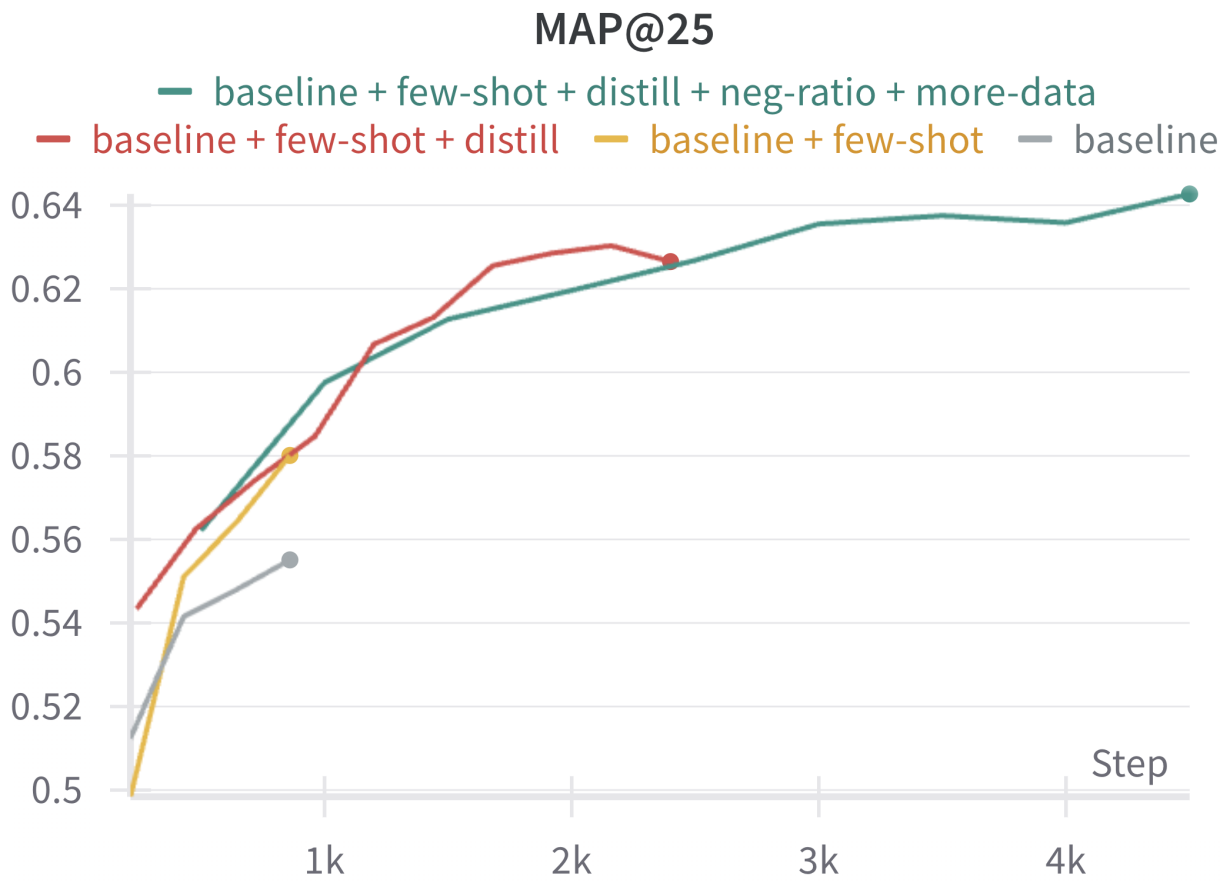
最後に、fine-tuneしたCoT reasoner（4.2節）を学習とinferenceのpipelineに加えました。学習時には、fine-tuneした`Qwen/Qwen2.5-14B` reasonerが生成したCoTを任意でmodel入力に追加しました。学習データの50%はCoTを使い、50%は使いませんでした。これにより、外部のreasoningがあるときはそれを使い、ないときは自身の内部reasoningに頼るようmodelを促しました。CoTにより、14B rerankerのprivate LB scoreは0.596から0.615へ向上しました **(+0.019)**。

これらの戦略の効果を次の表にまとめます。

Ablation (Qwen/Qwen2.5-14B)

Exp	Model	CV	LB (Public)	LB (Private)
1	Qwen/Qwen2.5-14B (Baseline)	0.555	0.545	0.495
2	Qwen/Qwen2.5-14B (Baseline + Few Shot)	0.580	0.559	0.531
3	Qwen/Qwen2.5-14B (Baseline + Few Shot + Distillation)	0.626	0.601	0.575
4	Qwen/Qwen2.5-14B (Baseline + Few Shot + Distillation + Negative Ratio + Extra Data)	0.642	0.608	0.596
5	Qwen/Qwen2.5-14B (Baseline + Few Shot + distillation + Negative Ratio + Extra Data + CoT)	**0.646**	**0.611**	**0.615**

fine-tuning中のlive evalでも、14B rerankerの性能が一貫して改善していることが示されました。

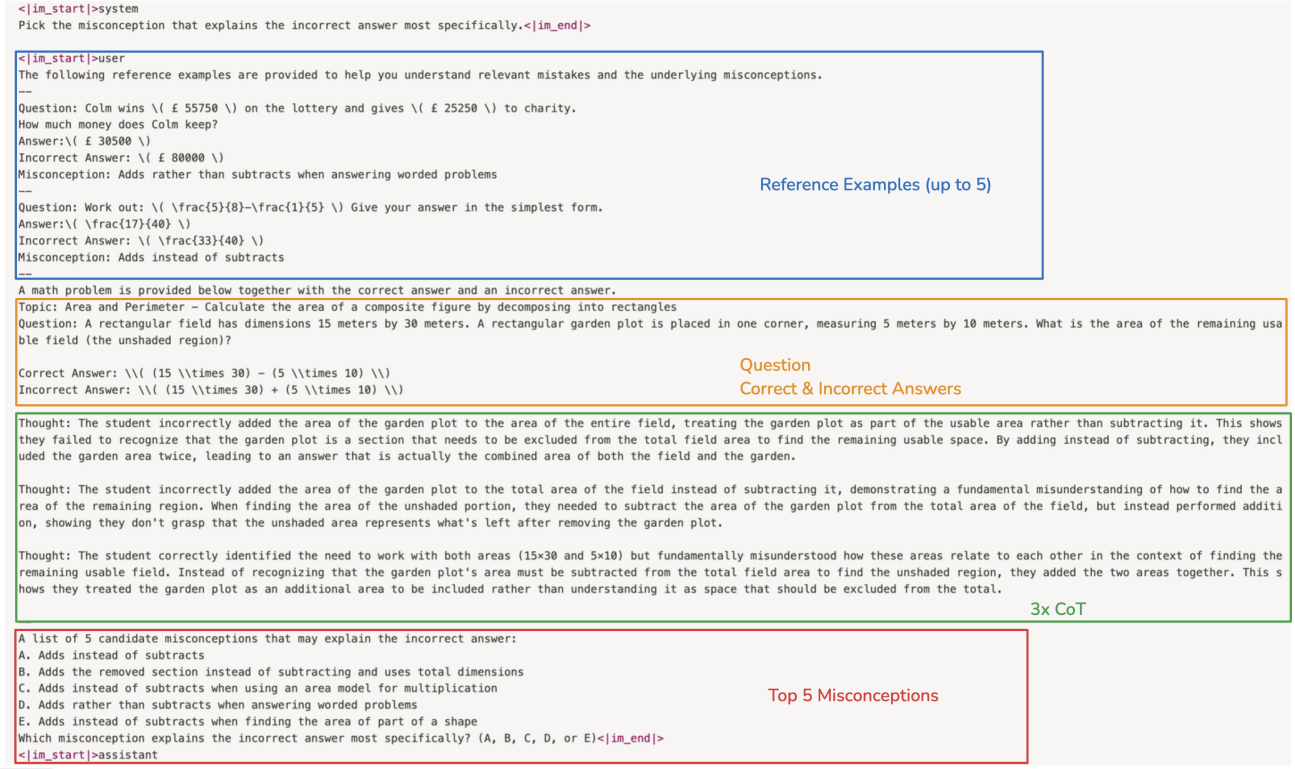


6.2 Pointwise Re-Ranker (`Qwen/Qwen2.5-32B`)

`Qwen/Qwen2.5-32B` modelは、14B rerankerとまったく同じ方法で（seedだけ変えて）fine-tuneしました。最良の32B modelのvalidation scoreは`0.663`で、14B rerankerの`0.646`を上回りました。inference時には、32B rerankerで14B rerankerの上位8候補を処理し、上位5候補に絞り込みました。32B rerankerを組み込むことで、private LB scoreは`0.615`から`0.625`へ向上しました **(+0.010)**。

6.3 Listwise Re-Ranker (`Qwen/Qwen2.5-72B`)

上位5候補は、最終的にfine-tuneした`Qwen/Qwen2.5-72B` modelによってlistwiseに順位付けしました。ここでは、modelが上位5候補すべてを一度に見ます。model入力の例を以下に示します。



このmodelは、pointwise re-rankerよりもさらに多くの情報にアクセスできました。すなわち、

- inference時に3つのChain of Thought (CoT)を見ます。7B、14B、32Bのreasonerから1つずつです
- training setから最大5個の参照例を見ます。5個の候補misconceptionそれぞれにつき1個です
- context windowの中ですべての候補を一度に見ます。

このmodelは、すべてのrerankerの中で最も良い`MAP@5`を達成しました（validation MAP@5は`0.661`）。LLMベースのlistwise rankerは、一般に候補の位置によるbiasに悩まされます。これを緩和する方法の1つは、候補misconceptionの位置をshuffleして複数回inferenceを実行することです（Test time augmentation (TTA)のような効果）。時間の制約のため、inference時にTTAを実行することはできませんでした。72B rerankerを組み込むことで、private LB scoreは0.625から0.638へ向上しました **(+0.013)**。

6.4 Quantization

fine-tuneしたrankerはAutoAWQでquantizeしました。性能劣化を抑えるため、各modelにタスク固有のcalibration datasetを使いました。32B rerankerの1つで、calibration datasetの影響を検証しました。結果を以下に示します。

Model	MAP@25	Top 1 Accuracy
Qwen/Qwen2.5-32B (bf16)	0.654	0.525
Qwen/Qwen2.5-32B (AWQ - Pile as calibration)	0.649	0.518
Qwen/Qwen2.5-32B (AWQ - Task dataset as calibration)	0.652	0.526

6.5 What didn't work

- Continuous Fine-tuning Without Loss Using Lora and Mergekitで説明されているmerging戦略

- Qwen/QwQ-32B-Preview: `Qwen/Qwen2.5-32B`と同じ手順で素朴にfine-tuneしましたが、結果は悪化しました。QwQ modelの適切な使い方を見つけるには、おそらく追加の研究が必要です。

7 Links

- Training code
- Inference Notebook
- Dataset with Synthetic + Competition MCQ
- CoT Dataset

8 References

- Novice Learner and Expert Tutor: Evaluating Math Reasoning Abilities of Large Language Models with Misconceptions
- vllm
- How To ~~Train~~ Synthesize Your ~~Dragon~~ Data
- FlagEmbedding
- AutoAWQ

補足Q&A

Dipam Chakraborty — 質問

多くの人が、LM HeadのYes/No確率をbinary classifierとして使っているのを見かけます。fine-tuningの際、それが最善のアプローチなのか、それとも新しいclassifier headを追加する方が良いのか、何かお考えはありますか。私は両方試し、classifier headの方がうまくいきましたが、この2つのアプローチを本格的に比較したのは初めてでした。お考えを共有していただけるとありがたいです。また、これほど非常に詳細な記事をありがとうございます。多くを学びました。

Andrey Galichin — 質問

非常に示唆に富む、美しいsolutionですね。ありがとうございます。

私たちもコンペの初期段階でgpt-4oを使ってChain-of-Thoughtの生成を試みましたが、modelの考え方を正しく定式化できず、また（そうしないよう促したにもかかわらず）modelが応答の中でmisconceptionを明示的に言及してしまうことがかなり頻繁にありました。

prompt作成の際にこの問題に遭遇しませんでしたか。もし遭遇したなら、具体的にどのようなpromptの修正が解決に役立ったのか教えていただけますか。

Raja Biswas — 回答

ありがとう、Dipam！

実は、私はmodelの大半を新しいclassification headで学習しており、後半の段階でYes/No tokenのアプローチに切り替えました。主な理由は、vllmのinference + 組み込みのprefix cachingを活用するためです。ここでは、Yes/Noのアプローチとclassifier headはどちらも同程度のscoreでした。

Raja Biswas — 回答

はい、私もClaudeがCoTの冒頭でmisconceptionの名前を言及することに気付きました。prompt engineeringによって、LLMに次の順序に従うよう求めることで修正しました。

- まず正しい計算を示す
- 誤りを示す、誤った計算を示す
- 生徒のreasoningにおける具体的な欠陥を説明する
- misconceptionがどのようにしてこの特定の誤りにつながったかを示す
- 関連するmisconceptionと区別する

xyzlsy — 質問

training setと合成datasetを1つにまとめてから5-fold splitを行ったのですか。それとも、training setに対して5-fold splitを行ってから、合成データをtraining setに直接追加したのですか。

Raja Biswas — 回答

training setに対して5-fold splitを行ってから、合成データをtraining setに直接追加しました。

xyzlsy — 質問

あなたのrecall modelは、model 1、model 2、model 3を組み合わせて最後のtokenをconcatenateすることで作られているのか、それとも別の方法を使っているのかを知りたいです。

Nikhil Tumbde — 質問

@conjuring92さん、詳細な説明をどうもありがとうございます。読み解くべきことがかなりたくさんあります。学びの一環として、あなたのアプローチについての私の理解を要約しようとしています。ただ、いくつか質問があり、あなた（またはforumの誰か）がより明確に理解する手助けをしてくださると本当にありがたいです。

質問

1. retrieverのfine-tuningには、いくつの負例を使いましたか。
2. 「各training batchにmisconceptionごとのdemonstrationが1つだけ現れるようにしたこと。複数のdemonstrationがあると、in-batch negativeを通じてlabelノイズが入り込むためです」 について詳しく説明していただけますか。1つのmisconceptionが、1つのbatch内で正解labelとして2回現れないという意味でしょうか。
3. retriever model 4 / 3の負例はどのように得たのですか。事前学習済みのembedding modelを使ったのですか。
4. re-rankerのdistillationの部分を説明していただけますか。
5. AWQ - calibrationにはどのdatasetを使いましたか。

> **要約**

1. Data Generation

a. 似たmisconceptionでclusterを作る

b. cluster内のmisconceptionと関連する4〜5個の問題をcontextとして与え、各clusterについて5個のMCQを（Claude 3.5 sonnetで）生成する。（データの品質は疑わしい場合がある）

c. GPT-4oを使って、生成したMCQの品質を0〜10のscaleで評価し、悪いものを除外する。

d. step bで追加のmisconception（misconception_mappingにないもの）が得られた。衝突を避けるため、与えられたmappingのものと似ているmisconceptionを（embedding modelのvectorを使って）除外した。

この時点で、作者は元のmisconception clusterと、misconceptionが紐付いた生成MCQ（元のものと生成したものの両方）を持っている。

2. COT Distillation

全体として、ここでのideaはClaudeの知識をより小さなmodelへdistillすることである。そのため、Claudeから生成を得て、それを使って小さなmodelをfine-tuneする。

a. 問題のcontext、misc、関連miscをpromptに入れ、Claudeを使って（元のtrain datasetに対する）COTを生成する。

b. 生成したtextと同じpromptで3つのmodel（7, 14, 32B）をfine-tuneするが、misconceptionと関連miscは除く（inference時には存在しないため）

この時点で、作者は問題のcontextだけ（misconceptionなし）から各不正解のCOT reasonを生成できる3つのmodel（reasoner）を持っている。

3. Models - retriever

a. step 1.dのデータ全体（1 foldを除く）で、MNR lossを使い12 epochでretrieverをfine-tuneする。{ここに少し混乱があるので、明確になったら更新します}

b. fine-tuneしたqwen-14Bを、step 1.bの合成データでpre-trainingするとrecallが向上した。

4. Model –reranker

全体として、2つのpointwise reranker（各misconceptionに対してyes/noを出力）と1つのlistwise rerankerを使用。

a. Pointwise rerankers (14/32B)

i. 合成dataset（1.dから）のpseudo labelは、train datasetでfine-tuneしたmodel（Qwen/Qwen2.5-Math-72BとQwen/Qwen2.5-72B）を使って生成した。

ii. pointwise rerankerのfine-tuningにはdataset全体を使った。training datasetの50%にはCOTも追加した。

iii. その他の詳細-

1. promptにはfew-shot exampleが含まれる。
2. re-rankerは、'yes'と'no'のtokenのlogitの差を予測として、cross entropy lossで学習した。
3. 各batchで1個の正例と(BS – 1)個の負例を取る。正例:負例の比は1:24。
4. fine-tuning中、LM headはfreezeした。

> **b. Listwise reranker**
>
> i. 上位5個の選択肢から1つを予測する。
>
> ii. {学習戦略は共有されていない}
>
> **5. Quantization**
>
> a. Fp16でfine-tuningを行い、modelはAWQでquantizeした。
>
> b. 使用したcalibration dataset {ここに少し混乱があるので、明確になったら更新します。}

Raja Biswas — 回答

Nikhilさん、ご質問ありがとうございます。以下に私の回答を記します。

> retrieverのfine-tuningには、いくつの負例を使いましたか。

標準的な**in-batch negatives**の設定を使いました。したがって、batch_sizeが128なら、各query（問題 + 不正解）について127個の負例と1個の正例がありました。

> 「各training batchにmisconceptionごとのdemonstrationが1つだけ現れるようにしたこと。複数のdemonstrationがあると、in-batch negativeを通じてlabelノイズが入り込むためです」について詳しく説明していただけますか。1つのmisconceptionが、1つのbatch内で正解labelとして2回現れないという意味でしょうか。

例えば、あるbatchで正例が`(q1, m1), (q2, m2)..., (q128, m128)`だとします。in-batch negativesの設定では、これは`q1`が`m1`を正例、`[m2, m3, ..m128]`を負例として持つことを意味します。同様に、`q2`については`m2`が正例、`[m1, m3, ...m128]`が負例になります。学習中は、すべてのqueryとmisconceptionの組み合わせ（`128x128`の行列）についてsimilarity scoreを計算します。対角要素が正例に対応し、非対角要素は負例であるべきです。このコンペでは、queryからcontentへの対応がmany-to-oneの関係でした。そのため、randomなbatching戦略では、同じmisconceptionに属する複数のqueryが1つのbatchに入ってしまうケースが生じます。この場合、非対角要素にfalse negativeが含まれることになり、ノイズの多い、最適でない収束につながります。

> retriever model 4 / 3の負例はどのように得たのですか。事前学習済みのembedding modelを使ったのですか。

in-batch negativesを使っただけです。ただし、fine-tuneしたretrieverから得たmined hard negative（+ re-rankerの予測でde-noiseしたもの、Reference）も試しましたが、long tailのrecallを改善するには効果がありませんでした。

> re-rankerのdistillationの部分を説明していただけますか。

FlagEmbedding repoのdistillation lossを参照してください。

> AWQ - calibrationにはどのdatasetを使いましたか。

基本的に、re-rankerのtraining datasetのsubsetをcalibration datasetとして使いました。

Raja Biswas — 回答

これらのmodelのsimilarity scoreを平均しました。

Yannan Chen — 質問

この詳細なsolutionをどうもありがとうございます。細部にまで注意を払っていること、そして限られたGPUリソースをフルに活用してこれほど何段階ものrerankingを行っていることに、とても感銘を受けました。お聞きしてもよいでしょうか。

- quantizationに「recovery adapterを追加する」と書かれていましたが、それは何ですか。最終的に適用したのですか。
- 72B modelのinferenceを2台のT4 GPUに収めるための重要なtrickは何ですか。
- コンペ中、合計実行時間はどれくらいに収めることができましたか。コンペ終盤に時間切れやout of memoryを経験しましたか。

改めてありがとうございます。1位おめでとうございます。このsolutionはまさにそれにふさわしいものです。

Raja Biswas — 回答

温かいお言葉をありがとうございます。あなたもsolo goldおめでとうございます！🎉

> quantizationに「recovery adapterを追加する」と書かれていましたが、それは何ですか。最終的に適用したのですか。

このideaはApple Intelligence report（Section 5.2）から来ています。

> … quantizeしたmodelは、多くの場合ある程度の品質低下を示します。そのため、quantizeしたmodelを機能開発のためにapplication teamへ直接渡すのではなく、品質回復のためにparameter-efficientなLoRA adapterのセットを取り付けます。これらのLoRA adapterの学習レシピが、pre-trainingおよびpost-trainingのプロセスと一貫するようにしています。

このコンペでは試せませんでした。

> 72B modelのinferenceを2台のT4 GPUに収めるための重要なtrickは何ですか。

主に@jagatkiranさんのこの素晴らしい記事に従いました。72B modelについて特別なinferenceのtrickはありませんでした。14B rankerについては、prefix cachingが鍵でした。

> コンペ中、合計実行時間はどれくらいに収めることができましたか。コンペ終盤に時間切れやout of memoryを経験しましたか。

最終的に選択した最良のsubmissionは~510分でした。OOMには遭遇しませんでしたが、終盤に何度か時間切れがありました…

ありがとうございます！

Geremie Yeo — 質問

Rajaさん、詳細なsolutionをありがとうございます。soloでの優勝おめでとうございます🥇

いくつか質問してもよいでしょうか。

わずか3か月という短期間に多くの実験を試したのは素晴らしいです。modelのfine-tuneには、どのようなGPUスペック/clusterを使いましたか。

背景: 私はRunPodでH100を借りていましたが、チームの外部データを使うと、14BのYes/No rerankerをLoRAでfine-tuneするのに > 5時間かかりました。初期の実験ではhard negativeの数をいろいろ（1,2,4,8,16,24）試しましたが、これら6つのmodelを学習するだけで合計2日以上かかってしまいます（1つのideaのためだけにです！）。終盤には32Bをfine-tuneしましたが、負例わずか4個での学習が1回の実験で > 20時間かかりました（外部データなしなら5時間）！ SFTTrainerを使って2 epoch学習しました。もっと速い方法があるのではと感じました…

実験の選び方について何かコツはありますか。

背景: MCTSとEEDIのコンペを通して、問題を解くためのideaはたくさん思い付けることに気付きました。しかし、どれを優先するかを決める力は改善が必要だと感じました。例えばEEDIでは、rerankingの性能向上を期待して、Qwen用のpromptを最適化するためにTextGradのカスタマイズに3日を費やしました（これと同様のものです）。しかし改善はごくわずかで、丸3日の労力に見合わなかったかもしれません…経験を積めば、この点はうまくなるのかもしれません 😅

最初から正しいアプローチを見つけ、3か月かけてそれを最大限に活かしたのは本当に刺激になります 👍

Raja Biswas — 回答

こんにちは、bogoconic1さん！

どうもありがとうございます。あなたは素晴らしい道のりを歩んでいて、さまざまなタイプのコンペで好成績を収めていますね。

> わずか3か月という短期間に多くの実験を試したのは素晴らしいです。modelのfine-tuneには、どのようなGPUスペック/clusterを使いましたか。

vast.aiの2xH100 GPUを使いました。modelはHF accelerateで学習し、いくつかの基本的な高速化を施しました。例えば、(1) torch.compile、(2) flash-attention、(3) QLoRAではなくLoRA、(4) sequence lengthを8の倍数にpaddingする、です。私の環境でも、14B re-rankerのfull runには~5時間かかっていました。しかし、full runをすることはほとんどありませんでした。(a) 学習の~50%で、実験の良し悪しの見当が十分につきました。(b) 実験は主に7B modelで行いました。32Bについては、14Bで最も良かった設定をそのままコピーしただけです。私の意見では、実験のsetupをできる限り軽量に保つことが鍵です。

> 実験の選び方について何かコツはありますか。

私もこれにはよく苦労します。explorationとexploitationのバランスを正しく取るのは難しいです。万能の解決策はありませんが、NLPのコンペでは次の点が役立つと思います。

1. fine-tuneの起点となる最良のbackboneの組を見つけるよう努める
2. 良いbackboneが見つかり、まずまずのbaselineができたら、データ関連の実験に集中する
3. 各実験の後にtakeawayを書き留める。関連するplot、error analysisなども含める。これが次に何をすべきかを教えてくれるはずです
4. distillation/pseudo labelingのような手法は、ほぼ常に性能と汎化に役立つ

明日ほかの点を思い付いたら、このリストに追加します。これらが当たり前すぎるのか、それとも非常に主観的なのかはよくわかりませんが…

Yannan Chen — 質問

ご説明ありがとうございます。また、retrieverでlora_aとlora _bに異なるlearning rateを使っていることにも気付きました。これは興味深い細部です。その背後にある根拠を伺ってもよいでしょうか。

Atharva Ingle — 質問

詳細なwriteupをありがとうございます。学ぶことがたくさんあります。次の部分で少し混乱しています。

質問: causal LMをfine-tuneしてYes/No tokenを出力させているのに、なぜlm_headをfreezeしたのですか。その背後にある理由は何でしょうか。lm_headにはYesかNoのtokenに高い確率を持たせたいので、freezeするとこのmappingを学習できなくなってしまいます。この部分を説明していただけますか。

> logitは、次のように'Yes'と'No'のtoken scoreの差を取って計算しました。

```
outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
logits_yes = outputs.logits[:, -, .yes_loc]
logits_no = outputs.logits[:, -, .no_loc]
logits = logits_yes - logits_no
logits = logits.reshape(-, .group_size)
labels = labels.to(logits.device).reshape(-)
ce_loss = .loss_fn(logits, labels)
```

> 各batchには、与えられたQuestion-Incorrect Answerの組み合わせについて、1個の正例misconceptionとbatch_size-1個の負例misconceptionを含めました。labelは正例misconceptionのindexとし、学習にはcross entropy lossを使いました。modelはLoRAで学習し、設定はr=64, alpha=128, LoRA on all linear layersとしました。fine-tuning中、LLM（Qwen/Qwen2.5-14B）のlanguage modelling head（lm_head）はfreezeしました。

YuhaoWu123 — 質問

洞察を共有していただき、本当にありがとうございます。非常に興味深く、しっかりしたsolutionです。ただ、promptの設計についていくつか確認したい質問があります。

1. 提供したmisconceptionは互いに非常に似ているとのことですが、1つの問題（Q）に対して、3つの不正解（A）はまったく異なるタイプのmisconceptionに対応している可能性があります。その場合、各referenceは1つのQAペアに対応するのでしょうか。それとも1つのQが3つの異なるAの選択肢に対応するのでしょうか。
2. promptでは、生成されるsampleのmisconceptionのタイプが新しいものでもよいとは明示されていません。しかし、まずどのmisconceptionを利用する必要があるかを特定するよう述べており、これは提供されたmisconceptionのうち、reference questionでカバーされていないものを選ぶことを意味しているように思えます。もしそうなら、構築したdatasetに新しいタイプのmisconceptionが現れることをどのように保証できるのでしょうか。
3. promptでは、 タグの中で各不正解がどのmisconceptionに対応するかを示す必要があると述べています。これは、reference questionには不正解だけが与えられ、misconceptionは与えられないという意味でしょうか。
4. 上記の質問の多くは、完全なpromptにおける タグと タグの中身がよくわからないことから来ています。これらのタグを含む、完全に構築されたpromptの例を提供していただけないでしょうか。改めてありがとうございます！〜〜〜

（訳注: 原文では3と4のタグ名が欠落しています。）

Miles — 質問

優勝おめでとうございます！ 72Bからのdistillationなしで、合成データを使って14B rerankerを学習することは試しましたか。どのような性能になるのか気になります。

Raja Biswas — 回答

LoRA+の論文 LoRA+: Efficient Low Rank Adaptation of Large Models に従いました。この論文は、LoRAのAとBに異なるLRを使うことで、収束が速くなり性能も向上することを示しています。

> …Hu et al. (2021)で最初に導入されたLow Rank Adaptation (LoRA)は、width（embedding次元）の大きなmodelに対して最適でないfinetuningをもたらします。これは、LoRAのadapter行列AとBが同じlearning rateで更新されることに起因します。width の大きなnetworkに対するscalingの議論を用いて、AとBに同じlearning rateを使うと効率的なfeature learningができないことを示します。次に、LoRAのこの非最適性は、LoRAのadapter行列AとBに、適切に選んだ比率で異なるlearning rateを設定するだけで修正できることを示します。この提案アルゴリズムをLoRA+と呼びます。大規模な実験において、LoRA+はLoRAと同じ計算コストで、性能（1-2 %の改善）とfinetuningの速度（最大 ~ 2Xの高速化）を改善します。

Raja Biswas — 回答

興味深いですね。良い実験になるはずです。ただ、合成データのmisconceptionはcuration後でもまだノイズを含んでいる（コンペのデータより品質が低い）かもしれないと考えて、これは試しませんでした…

Raja Biswas — 回答

@atharvaingleさん、`lm_head`をfreezeすることについてです。

- `logit(y=Yes) = features . W_lm_head_yes` : Yes/No tokenのlogitは、Yes/No tokenの位置にあるLM headの重みと、sequence末尾におけるmodelのhidden stateの両方によって制御されます。LM headをfreezeしても、fine-tuning中に他のparameterを更新してfeatureがlossに従って動くようにすることで、networkはlogitを調整できます。
- prompt templateによって、（LM headをfine-tuneしなくても）Yes/No tokenのlogitはすでに高くなります。
- Qwen2.5のtokenizerには150k+のtokenがあり（そのうち関心があるのは2 tokenだけです）、LM Headをunfreezeすると必要なVRAMが大幅に増えます。
- ただし、いつものように、unfreezeとfreezeの両方のケースを試して性能差を見るべきです。直感的には、差は大きくないだろうと感じました。

Raja Biswas — 回答

promptの例を以下に示します。

```
You will be generating Multiple Choice Questions (MCQs)  diagnose specific mathematical
misconceptions. Here are the misconceptions you should focus on:

<misconceptions>
- Thinks  the perimeter  a square grid  the number  squares surrounding the rectangle
- Calculates area  asked  perimeter
- Counts a side length more than once  counting perimeter due  marking a starting point
- Thinks  the perimeter  a square grid  the number  squares inside  are touching the perimeter  the
shape
- Calculates perimeter  asked  area
- When calculating perimeter, only adds two sides
- Believes lines inside the shape are included  finding perimeter
- When calculating perimeter, only adds the marked lengths
- When calculating perimeter, multiplies instead  adds
```

```
- Thinks only labelled dimensions need  be included  calculating the perimeter
- Confuses perimeter  area
- Fails  use the properties  simple shapes  work out missing lengths
- Does  know  we can use the conservation  perimeter  find missing lengths
- Makes an assumption about line segments being equal within a shape
- Mixes up width  height  a rectangle
- Does  know how  find missing lengths  a composite shape
</misconceptions>

Here are reference MCQs  demonstrate how  effectively diagnose these misconceptions:

<reference_mcqs>
[{:,:,:,
```

> るように思えます。もしそうなら、構築したdatasetに新しいタイプのmisconceptionが現れることをどのように保証できるのでしょうか。

鋭い指摘です。考えられるmisconceptionを指定しても、LLMはそれでも追加のmisconceptionを生成したがります。このpromptに落ち着く前の、以前のpromptでは、さらに多くの新しいmisconceptionが生成されていました。

> promptでは、タグの中で各不正解がどのmisconceptionに対応するかを示す必要があると述べています。これは、reference questionには不正解だけが与えられ、misconceptionは与えられないという意味でしょうか。

完全なpromptの例をここに載せました。お役に立てば幸いです！

YuhaoWu123 — 質問

こんにちは、もう1つ質問させてください。この問題は最終的に正解のMAP@Kを使いますが、あなたのcodeは上位K個すべての答えについてMAP@Kを計算しています。codeのmetricが、問題で求められているmetricと一致していないのはなぜなのか気になります。私が言っているcodeは次のものです⬇️

```
    for i, p in enumerate(predicted):
        # first condition checks whether it is valid prediction
        # second condition checks if prediction is not repeated
        if p in actual and p not in predicted[:i]:
            num_hits += 1.0
            score += num_hits / (i + 1.0)
```

```
    / (len(actual), k)
```

お返事をお待ちしています。どうもありがとうございます！

YuhaoWu123 — 質問

こんにちは〜 もう1つ質問させてください。https://gist.github.com/rbiswasfc/8d2bfec5c2a358e8beeb2df390111f9d にあるあなたのnotebookでは、ranker_oof_df.csvという名前のfileを使っています。solutionの中で、「具体的には、まずvalidation dataに対するretriever/rankerの予測におけるmisconceptionの共起統計を利用して、密接に関連するmisconceptionのclusterを作りました」と述べています。私の質問は、ここで言う「validation data」とは、元のdataset全体をtraining setとvalid setに分割し、そのtraining setでmodelを学習して得たvalidation dataに対する予測という意味でしょうか。どうもありがとうございます！

Raja Biswas — 回答

@yuhaowu123さん、はい、そのとおりです。validation splitに対してretrieveした上位候補を使っています。

Raja Biswas — 回答

metricの計算は問題ないように見えます。metricについての詳細な記事はこちらです: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/534077